



Ejercicio sistema de información de una biblioteca.

Enlace a GITHUB: <https://github.com/MariaBuzarhon/AD-Biblioteca.git>

Inicio de sesión (LoginController.java):

Biblioteca los Olmos

Usuario:

Contraseña:

Mostrar contraseña

Iniciar sesión

```
@FXML
public void initialize() {
    botonIniciarSesion.disableProperty().bind(
        nombreUsuario.textProperty().isEmpty()
        .or(contraseñaUsuario.textProperty().isEmpty())
    );
    contraseñaVisible.textProperty().bindBidirectional(
        contraseñaUsuario.textProperty()
    );
    checkboxMostrarContraseña.selectedProperty().addListener(
        ( ObservableValue<extends Boolean> obs, Boolean oldVal, Boolean newVal)
        -> mostrarContraseña(newVal)
    );
}

private void mostrarContraseña(boolean mostrar) { 1 usage
    contraseñaVisible.setVisible(mostrar);
    contraseñaVisible.setManaged(mostrar);

    contraseñaUsuario.setVisible(!mostrar);
    contraseñaUsuario.setManaged(!mostrar);
}
```

Al iniciar la aplicación, se muestra la pantalla de inicio de sesión, donde el usuario debe introducir su correo electrónico y contraseña.

El método `initialize()` se ejecuta automáticamente al cargarse la vista. En él se controla que el botón *Iniciar sesión* permanezca deshabilitado mientras el nombre de usuario o la contraseña estén vacíos, evitando intentos de acceso incompletos.

Además, los campos de contraseña visible y oculta se encuentran sincronizados, lo que permite mostrar u ocultar la contraseña sin perder el texto introducido.

El checkbox permite alternar entre ambos modos.

```

@FXML
void iniciarSesion(ActionEvent event) {
    String email = nombreUsuario.getText();
    String contraseña = contraseñaUsuario.getText();
    UsuarioDAO usuarioDAO = new UsuarioDAO();
    Usuario usu = usuarioDAO.login(email, contraseña);

    if (usu == null) {
        mensajeErrorLogin.setText("Usuario o contraseña incorrectos");
        System.out.println("Usuario o contraseña incorrectos");
    } else {
        System.out.println("Bienvenido " + usu.getNombre());
        System.out.println("Tipo de usuario: " + usu.getTipo());

        try {
            if (usu.getTipo().equalsIgnoreCase("administrador")) {
                FXMLLoader loader =
                    new FXMLLoader(getClass().getResource("/fxml/menuAdminBiblio.fxml"));
                Parent root = loader.load();
                MenuPrincipalAdminController adminController = loader.getController();
                adminController.setUsuario(usu);
                Stage stage = new Stage();
                stage.setTitle("Menú Administrador");
                stage.setScene(new Scene(root));
                stage.show();
            } else {
                FXMLLoader loader = new FXMLLoader(
                    getClass().getResource("/fxml/menuBiblio.fxml")
                );
                Parent root = loader.load();
                MenuUsuarioNormalController menuController =
                    loader.getController();
                menuController.setUsuario(usu);
                Stage stage = new Stage();
                stage.setTitle("Menú Biblioteca");
                stage.setScene(new Scene(root));
                stage.show();
            }
            ((Stage) botonIniciarSesion.getScene().getWindow()).close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
}

```

El método `iniciarSesion(ActionEvent event)` se ejecuta cuando el usuario pulsa el botón *Iniciar sesión*.

En este método se obtienen los datos introducidos (correo y contraseña) y se valida que correspondan a un usuario registrado en el sistema.

- Si los datos no son correctos, se muestra un mensaje de error.
- Si el usuario es válido, se comprueba su tipo (administrador o usuario normal) y se carga el menú correspondiente.

Biblioteca

Biblioteca los Olmos

Usuario:

Contraseña:

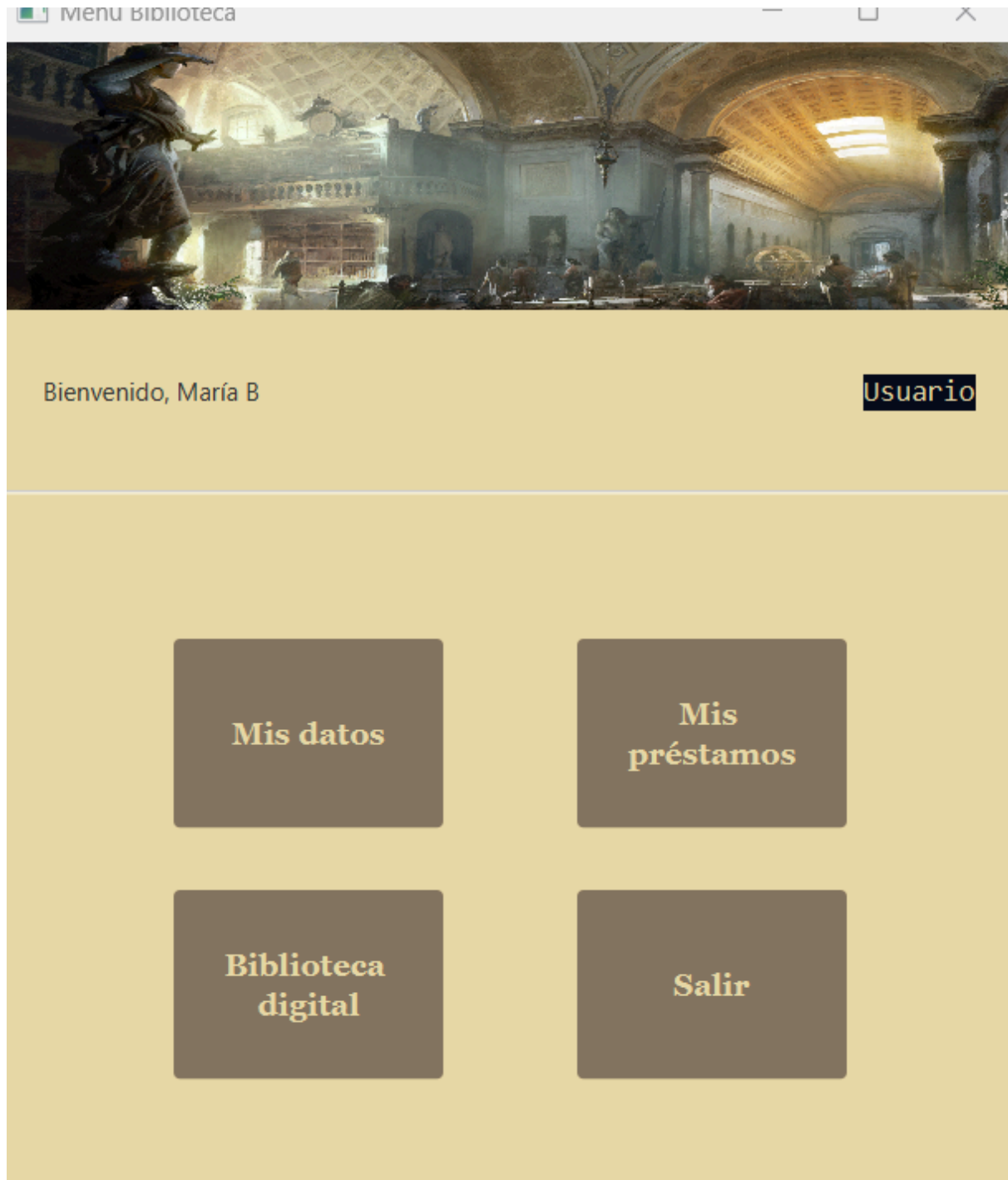
Mostrar contraseña ☐

Usuario o contraseña incorrectos

Iniciar sesión

Menú usuario normal:

Tras iniciar sesión como usuario normal, se muestra el menú principal desde el cual se puede acceder a las distintas funcionalidades disponibles.



Consultar datos del usuario:

Al hacer clic en la opción “**Mis datos**”, se accede a una pantalla donde se muestra la información personal del usuario.

```
private Usuario usuario;  1 usage

public void mostrarInformacion(Usuario usu) {  1 usage
    this.usuario = usu;

    labelBienvenida.setText("Datos de " + usu.getNombre());

    labelInfo.setText(
        "ID: " + usu.getId() +
        "\nNombre: " + usu.getNombre() +
        "\nDNI: " + usu.getDni() +
        "\nEmail: " + usu.getEmail() +
        "\nTipo de usuario: " + usu.getTipo()
    );
}

@FXML  1 usage
void volverAlMenu(ActionEvent event) {
    Stage stage = (Stage) botonVolver.getScene().getWindow();
    stage.close();
}
```

El método `mostrarInformacion(Usuario usu)` recibe el usuario que ha iniciado sesión y presenta en pantalla datos como:

- ID del usuario
- Nombre
- DNI

- Email
- Tipo de usuario



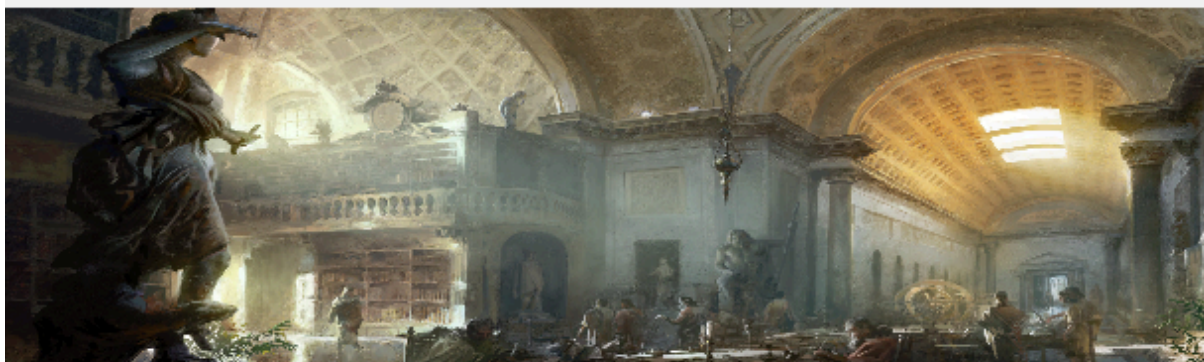
Datos de María B

Volver

ID: 5
Nombre: María B
DNI: 45698745L
Email: mariab@example.com
Tipo de usuario: normal

Consultar préstamos del usuario:

Al hacer clic en “**Mis préstamos**”, se muestra una lista con todos los préstamos asociados al usuario.



Préstamos de María B

Volver

ID Préstamo	ID Ejemplar	Nombre Ejemplar	Fecha Inicio	Fecha Devolución
4	9	La Iliada	2026-01-06	2026-01-06
5	7	1984	2026-01-06	2026-01-06
6	6	1984	2026-01-06	2026-01-06

Devolver


```

@FXML
public void initialize() {

    colIdPrestamo.setCellValueFactory(
        new PropertyValueFactory<>("id")
    );

    colIdEjemplar.setCellValueFactory( CellDataFeatures<Prestamo, Integer> cellData ->
        new SimpleIntegerProperty(
            cellData.getValue().getEjemplar() != null
                ? cellData.getValue().getEjemplar().getId()
                : 0
        ).asObject()
    );

    colNombreEjemplar.setCellValueFactory( CellDataFeatures<Prestamo, String> cellData -> {
        if (cellData.getValue().getEjemplar() != null &&
            cellData.getValue().getEjemplar().getIsbn() != null) {
            return new SimpleStringProperty(
                cellData.getValue().getEjemplar().getIsbn().getTitulo()
            );
        } else {
            return new SimpleStringProperty("Sin título");
        }
    });

    colFechaInicio.setCellValueFactory(
        new PropertyValueFactory<>("fechaInicio")
    );

    colFechaDevolucion.setCellValueFactory(
        new PropertyValueFactory<>("fechaDevolucion")
    );
}

```

En el método `initialize()` se definen las columnas del `TableView` y los datos que muestra cada una.

Se realizan comprobaciones para evitar errores en caso de que algún dato no esté disponible, mostrando valores por defecto cuando es necesario.

En este caso, se puede observar que los tres préstamos existentes se encuentran devueltos, por lo que el usuario puede solicitar un nuevo préstamo.

Biblioteca digital:

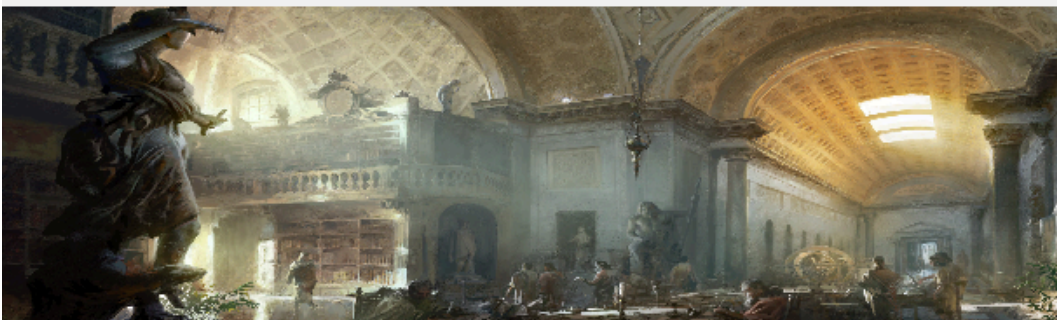
En esta pantalla se muestran los ejemplares disponibles para préstamo en la biblioteca.

En el método `initialize()` se definen las columnas del `TableView` que muestra la información de cada libro.

```
@FXML
public void initialize() {
    colEstado.setCellValueFactory( CellDataFeatures<Ejemplar, String> cell -> new SimpleStringProperty(cell.getValue().getEstado()));
    colIdLibro.setCellValueFactory( CellDataFeatures<Ejemplar, Integer> cell -> new SimpleIntegerProperty(cell.getValue().getId()).asObject());
    colTitulo.setCellValueFactory( CellDataFeatures<Ejemplar, String> cell -> new SimpleStringProperty(cell.getValue().getIsbn().getTitulo()));
    colIsbn.setCellValueFactory( CellDataFeatures<Ejemplar, String> cell -> new SimpleStringProperty(cell.getValue().getIsbn().getIsbn()));
    colAutor.setCellValueFactory( CellDataFeatures<Ejemplar, String> cell -> new SimpleStringProperty(cell.getValue().getIsbn().getAutor()));
}
```

El método `cargarLibrosDisponibles()` consulta los ejemplares disponibles mediante la clase `EjemplarDAO` y los muestra en la tabla.

```
private void cargarLibrosDisponibles() { 2 usages
    List<Ejemplar> disponibles = ejemplarDAO.obtenerDisponibles();
    tablaLibrosDisponibles.setItems(FXCollections.observableArrayList(disponibles));
}
```



Libros disponibles para María B

Volver

Stock	ID Libro	Nombre Ejemplar	ISBN	Autor
Disponible	1	El Quijote	97812345...	Miguel de Cervan...
Disponible	2	El Quijote	97812345...	Miguel de Cervan...
Disponible	3	El Quijote	97812345...	Miguel de Cervan...
Disponible	4	Cien Años de Soledad	97898765...	Gabriel García Má...
Disponible	6	1984	97811111...	George Orwell
Disponible	7	1984	97811111...	George Orwell
Disponible	9	La Iliada	8469465	Homero

Pedir prestado

El método `pedirLibroPrestado(ActionEvent event)` controla todo el proceso de préstamo y aplica las normas de la biblioteca:

- Se comprueba si el usuario tiene una penalización activa.
- Se verifica que el usuario no tenga ya 3 préstamos activos.
- Se valida que se haya seleccionado un libro de la tabla.

Si todas las condiciones se cumplen, se realiza el préstamo y se muestra un mensaje confirmando que se ha realizado correctamente.

```

@FXML
void pedirLibroPrestado(ActionEvent event) {
    if (usuario.getPenalizacionHasta() != null && usuario.getPenalizacionHasta().isAfter(LocalDate.now())) {
        mostrarAlerta("No permitido", "Tienes una penalización activa hasta " + usuario.getPenalizacionHasta());
        return;
    }

    int activos = prestamoDAO.contarPrestamosActivos(usuario);
    if (activos >= 3) {
        mostrarAlerta("Limite alcanzado", "Ya tienes 3 préstamos activos");
        return;
    }

    Ejemplar seleccionado = tablaLibrosDisponibles.getSelectionModel().getSelectedItem();
    if (seleccionado == null) {
        mostrarAlerta("Aviso", "Selecciona un libro para pedir prestado");
        return;
    }
    prestamoDAO.crearPrestamo(usuario, seleccionado);
    cargarLibrosDisponibles();
    mostrarAlerta("Éxito", "Libro pedido prestado correctamente");
}

```

Al pedir prestado *Cien años de soledad*, este libro desaparece de la lista de disponibles.

Si el usuario intenta solicitar más de tres préstamos activos, el sistema impide la acción y muestra un aviso.



Éxito

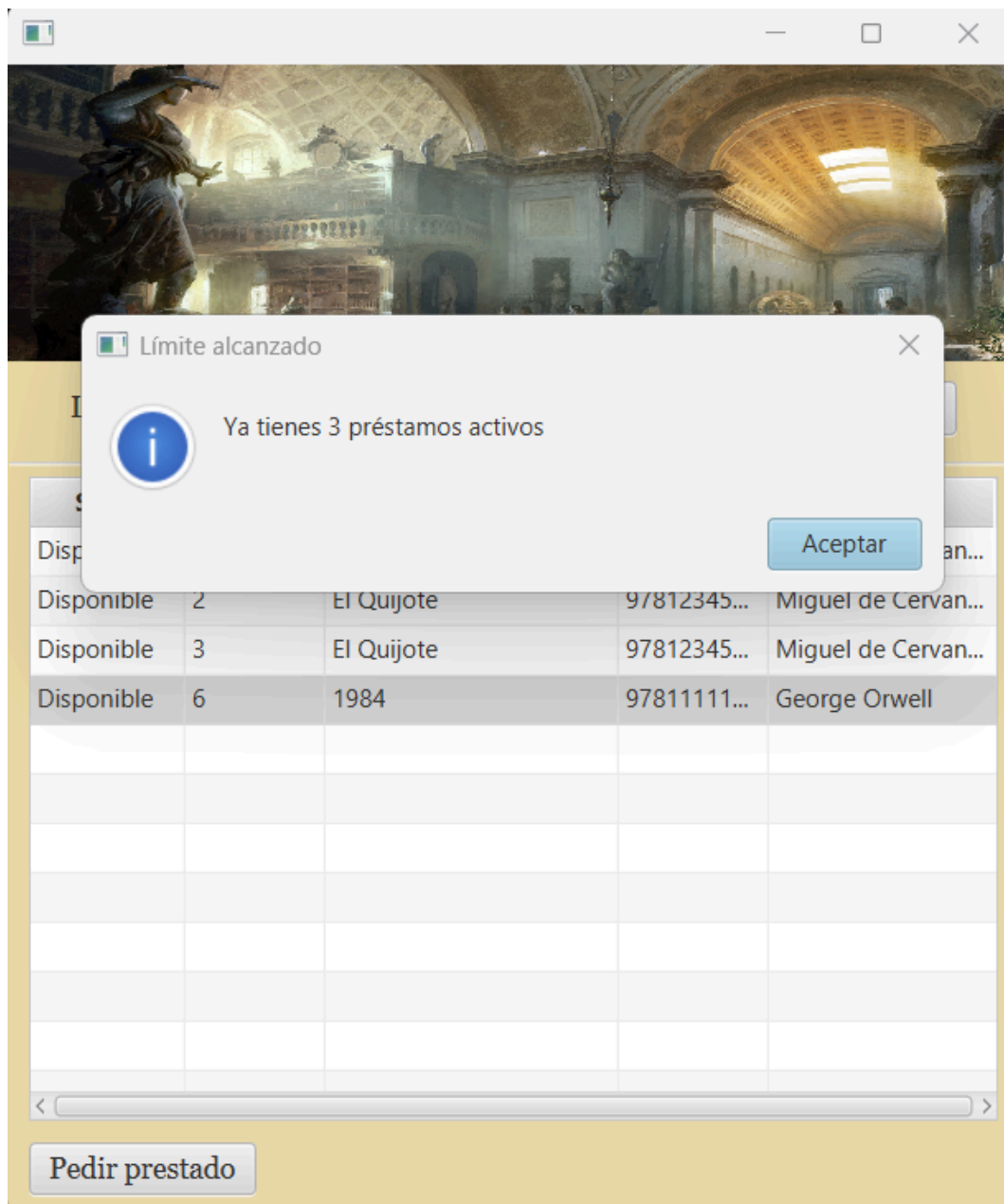
 Libro pedido prestado correctamente

Aceptar

Dispon	2	El Quijote	97812345...	Miguel de Cervan...
Dispon	3	El Quijote	97812345...	Miguel de Cervan...
Dispon	6	1984	97811111...	George Orwell
Dispon	7	1984	97811111...	George Orwell
Dispon	9	La Iliada	8469465	Homero

< >

Pedir prestado



Así que vamos a volver a nuestros préstamos y devolver los que acabamos de pedir:

—

□

×



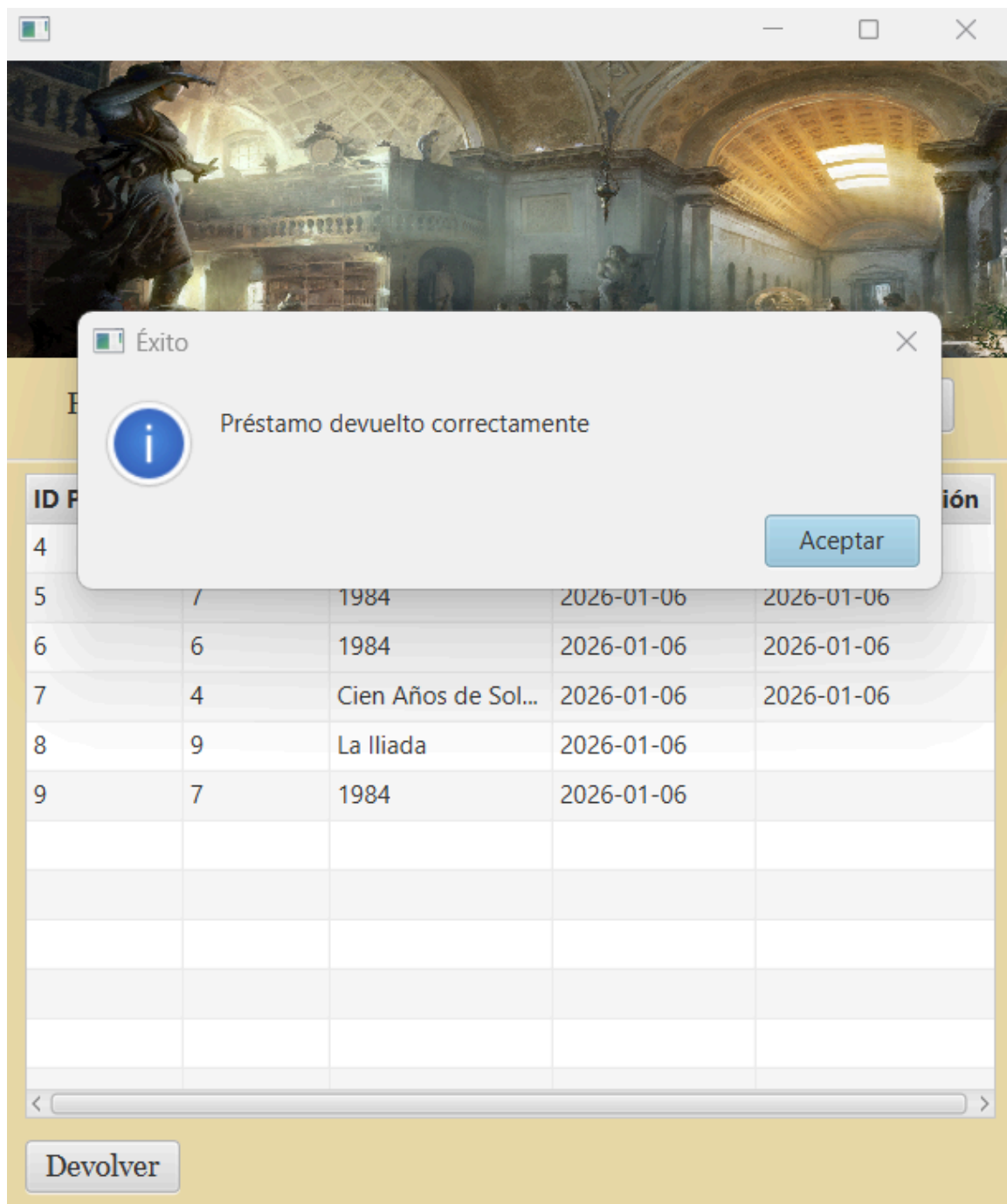
Préstamos de María B

Volver

ID Préstamo	ID Ejemplar	Nombre Ejemplar	Fecha Inicio	Fecha Devolución
4	9	La Iliada	2026-01-06	2026-01-06
5	7	1984	2026-01-06	2026-01-06
6	6	1984	2026-01-06	2026-01-06
7	4	Cien Años de Sol...	2026-01-06	
8	9	La Iliada	2026-01-06	
9	7	1984	2026-01-06	

< >

Devolver



Una vez devueltos todos:



Préstamos de María B

Volver

ID Préstamo	ID Ejemplar	Nombre Ejemplar	Fecha Inicio	Fecha Devolución
4	9	La Iliada	2026-01-06	2026-01-06
5	7	1984	2026-01-06	2026-01-06
6	6	1984	2026-01-06	2026-01-06
7	4	Cien Años de Sol...	2026-01-06	2026-01-06
8	9	La Iliada	2026-01-06	2026-01-06
9	7	1984	2026-01-06	2026-01-06

Devolver

Revisamos que vuelvan a aparecer en la lista de libros disponibles:



Libros disponibles para María B

Volver

Stock	ID Libro	Nombre Ejemplar	ISBN	Autor
Disponible	1	El Quijote	97812345...	Miguel de Cervan...
Disponible	2	El Quijote	97812345...	Miguel de Cervan...
Disponible	3	El Quijote	97812345...	Miguel de Cervan...
Disponible	4	Cien Años de Soledad	97898765...	Gabriel García Má...
Disponible	6	1984	97811111...	George Orwell
Disponible	7	1984	97811111...	George Orwell
Disponible	9	La Iliada	8469465	Homero

Pedir prestado

Como podemos ver están todos otra vez.

Menú administrador:

Cerramos la app y volvemos a iniciar sesión, esta vez como admin:

The image shows a web application window titled "Biblioteca". The main heading is "Biblioteca los Olmos" in a large, italicized serif font. Below the heading, there is a login form. The form has two input fields: "Usuario:" with the value "luis.lopez@example.com" and "Contraseña:" with the value "adminpass123". To the right of the password field, there is a checkbox labeled "Mostrar contraseña" which is checked. At the bottom of the form, there is a button labeled "Iniciar sesión".

Biblioteca

Biblioteca los Olmos

Usuario:

luis.lopez@example.com

Contraseña:

adminpass123

Mostrar contraseña ☒

Iniciar sesión

Tras iniciar sesión como administrador, se muestra el menú principal desde el cual se puede acceder a las distintas funcionalidades disponibles, las cuales son diferentes a las del usuario normal.



Lista de usuarios registrados:

En esta página podremos ver la información básica de todos los usuarios registrados.

En el método `initialize()` se definen las columnas del `TableView` que muestra los usuarios con su ID, nombre y correo electrónico.

```

@FXML
public void initialize() {

    colIdUsuario.setCellValueFactory( CellDataFeatures<Usuario, Integer> cell ->
    |     new SimpleIntegerProperty(cell.getValue().getId()).asObject()
    | );

    colIdNombre.setCellValueFactory( CellDataFeatures<Usuario, String> cell ->
    |     new SimpleStringProperty(cell.getValue().getNombre())
    | );

    colIdEmail.setCellValueFactory( CellDataFeatures<Usuario, String> cell ->
    |     new SimpleStringProperty(cell.getValue().getEmail())
    | );

    cargarUsuarios();
}

```

El método `cargarUsuarios()` obtiene la lista completa de usuarios mediante el `UsuarioDAO` y la convierte en una lista observable para mostrarla en la tabla.

Esto permite que cualquier cambio en la base de datos se refleje automáticamente al recargar la pantalla.

```

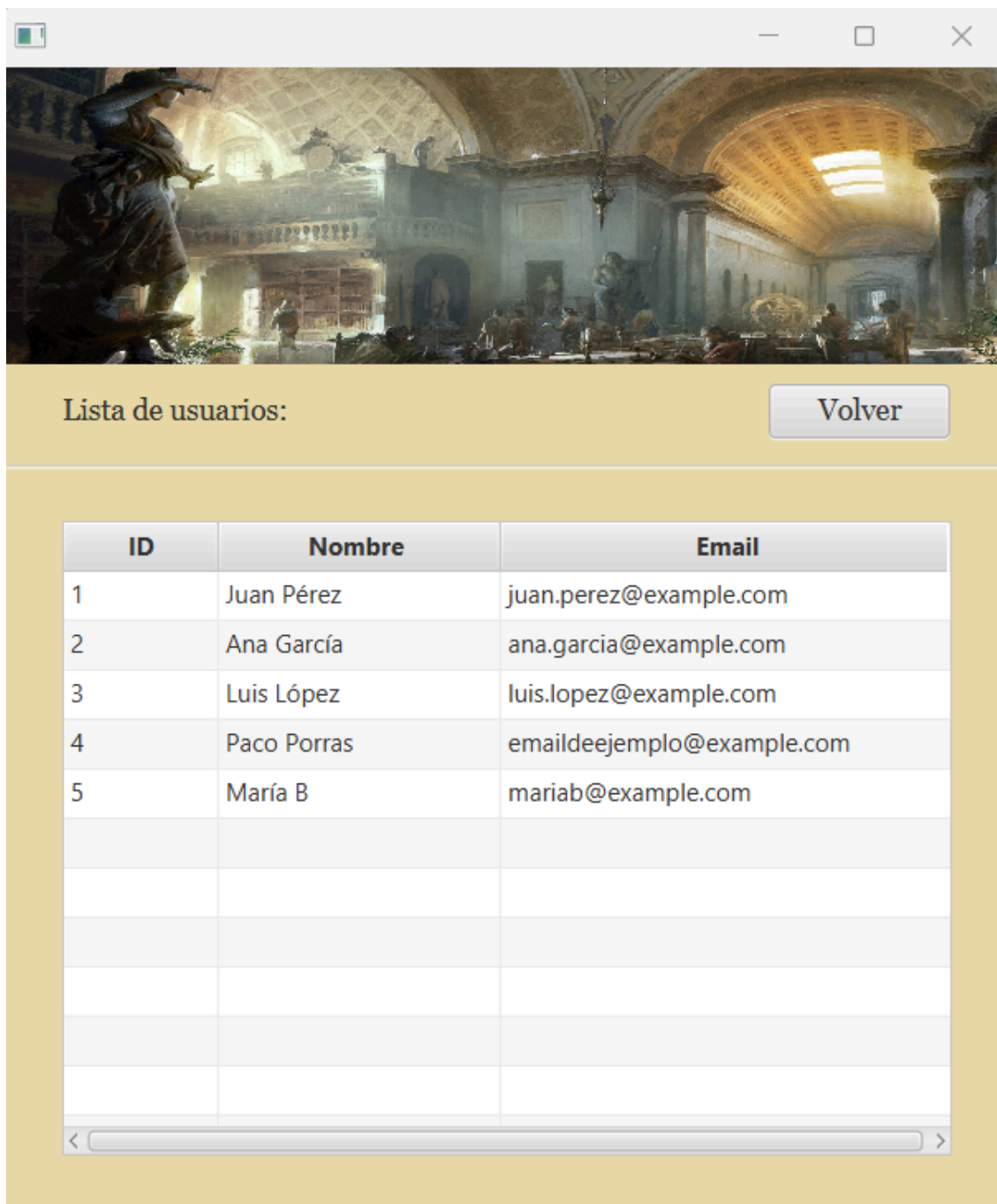
private UsuarioDAO usuarioDAO = new UsuarioDAO(); 1 usage
public void setUsuario(Usuario usuario) {
    |     this.usuario = usuario;
    |     cargarUsuarios();
    | }

```

```

private void cargarUsuarios() { 2 usages
    |     List<Usuario> usuarios = usuarioDAO.findAll();
    |     tablaListaUsuarios.setItems(
    |         FXCollections.observableArrayList(usuarios)
    |     );
}

```



Registrar usuario:

Esta clase permite al administrador agregar nuevos usuarios a la biblioteca, definiendo todos los datos necesarios y el tipo de usuario.

El método `registrarUsuario(ActionEvent event)` gestiona el proceso de creación de un usuario verificando que todos los campos (DNI, email, nombre, contraseña y tipo) estén completos. Si algún campo está vacío, se muestra un mensaje de alerta.

Comprueba que el tipo de usuario sea válido (`normal` o `administrador`) y crea un objeto `Usuario` con la información proporcionada.

Guarda el usuario en la base de datos mediante `UsuarioDAO` .

Muestra un mensaje confirmando que el usuario se ha registrado correctamente o un mensaje de error si, por ejemplo, el email ya existe.

Limpia los campos del formulario para permitir nuevos registros.

```
@FXML
void registrarUsuario(ActionEvent event) {
    if (tfDNI.getText().isEmpty() || tfEmail.getText().isEmpty() ||
        tfNombre.getText().isEmpty() || tfPassword.getText().isEmpty() ||
        tfTipo.getText().isEmpty()) {
        mostrarAlerta("Campos vacíos", "Debes rellenar todos los campos");
        return;
    }

    String tipo = tfTipo.getText().toLowerCase();

    if (!tipo.equals("normal") && !tipo.equals("administrador")) {
        mostrarAlerta("Tipo incorrecto", "El tipo debe ser 'normal' o 'administrador'");
        return;
    }

    Usuario nuevo = new Usuario();
    nuevo.setDni(tfDNI.getText());
    nuevo.setEmail(tfEmail.getText());
    nuevo.setNombre(tfNombre.getText());
    nuevo.setPassword(tfPassword.getText());
    nuevo.setTipo(tipo);

    boolean exito = usuarioDAO.guardarUsuario(nuevo); // método en UsuarioDAO

    if (exito) {
        mostrarAlerta("Éxito", "Usuario registrado correctamente");
        limpiarCampos();
    } else {
        mostrarAlerta("Error", "No se pudo registrar el usuario. ¿El email ya existe?");
    }
}
```

—□×

Éxito

×

 Usuario registrado correctamente

Aceptar

Usuario conectado: Luis López

Volver

DNI

45698712C

Nombre

Jane Doe

Email

janedoe@example.com

Contraseña

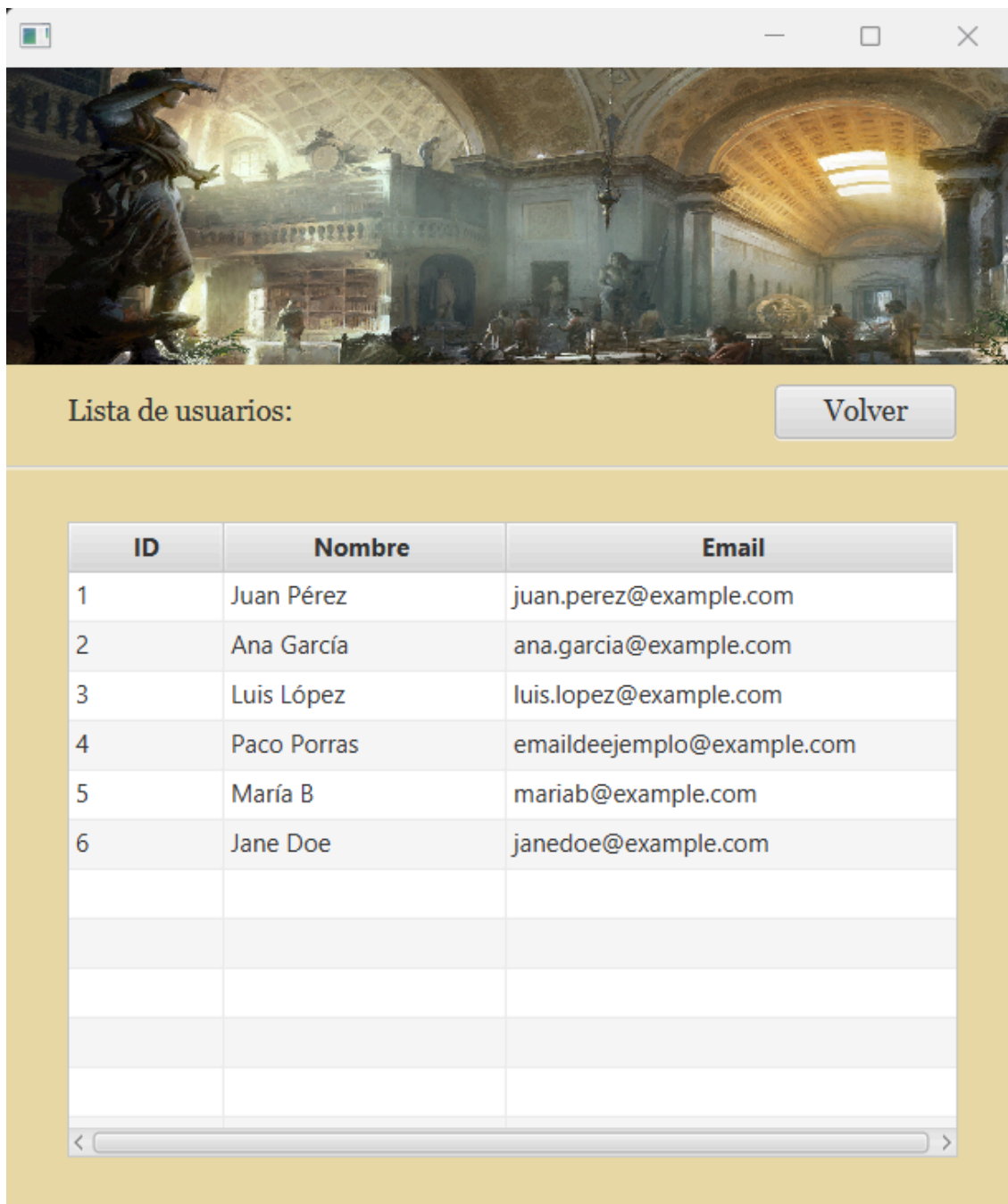
jane

Tipo

normal

Registrar

Cancelar



Inventario de libros:

Esta clase permite al administrador consultar todos los libros disponibles en la biblioteca, junto con información sobre sus ejemplares y disponibilidad.

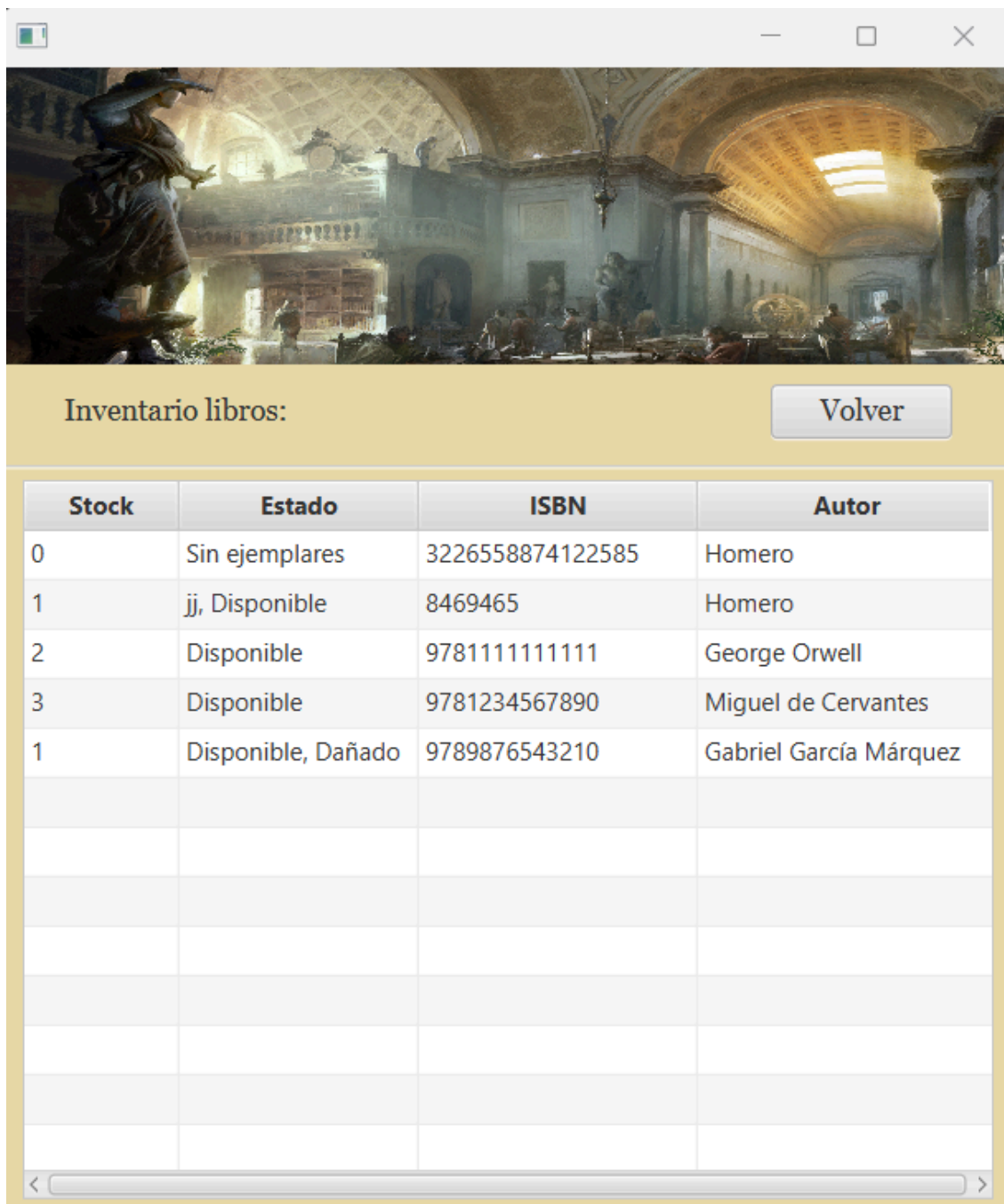
En el método `initialize()` se definen las columnas del `TableView`: ISBN, estado, autor y ejemplares disponibles.

Para las columnas de Estado y Ejemplares disponibles, se utiliza la lista de ejemplares asociada a cada libro obtenida mediante `EjemplarDAO`. Esto permite que la tabla muestre

información completa y actualizada sobre la disponibilidad real de cada título.

```
@FXML
public void initialize() {
    colIsbn.setCellValueFactory( CellDataFeatures<Libro, String> cell -> new SimpleStringProperty(cell.getValue().getIsbn()));
    colAutor.setCellValueFactory( CellDataFeatures<Libro, String> cell -> new SimpleStringProperty(cell.getValue().getAutor()));

    colEstado.setCellValueFactory( CellDataFeatures<Libro, String> cell -> {
        List<Ejemplar> ejemplares = ejemplarDAO.obtenerPorLibro(cell.getValue());
        if (!ejemplares.isEmpty()) {
            String estados = ejemplares.stream() Stream<Ejemplar>
                .map(Ejemplar::getEstado) Stream<String>
                .distinct()
                .collect(Collectors.joining(", "));
            return new SimpleStringProperty(estados);
        }
        return new SimpleStringProperty("Sin ejemplares");
    });
    colEjemplaresDisponibles.setCellValueFactory( CellDataFeatures<Libro, Integer> cell -> {
        List<Ejemplar> ejemplares = ejemplarDAO.obtenerPorLibro(cell.getValue());
        int disponibles = (int) ejemplares.stream()
            .filter( Ejemplar e -> e.getEstado().equalsIgnoreCase("disponible"))
            .count();
        return new SimpleIntegerProperty(disponibles).asObject();
    });
    cargarLibros();
}
```



Inventario libros:

Volver

Stock	Estado	ISBN	Autor
0	Sin ejemplares	3226558874122585	Homero
1	jj, Disponible	8469465	Homero
2	Disponible	9781111111111	George Orwell
3	Disponible	9781234567890	Miguel de Cervantes
1	Disponible, Dañado	9789876543210	Gabriel García Márquez

El método `cargarLibros()` obtiene la lista completa de libros mediante `LibroDAO` y la convierte en una lista observable para mostrarla en la tabla.

De este modo, cualquier cambio en la base de datos (como la adición de un libro o cambios en el estado de los ejemplares) se refleja al recargar la pantalla.

```
private void cargarLibros() { 1 usage
    List<Libro> lista = libroDAO.obtenerTodos();
    tablaLibros.setItems(FXCollections.observableArrayList(lista));
}
```

Registrar libros:

Esta clase permite al administrador agregar nuevos libros al catálogo de la biblioteca, introduciendo la información de cada título.

El método `registrarLibro(ActionEvent event)` gestiona el proceso de registro, tiene los datos introducidos por el administrador: ISBN, título, autor y ejemplares.

Se comprueba que todos los campos estén completados, si alguno estuviese vacío, se muestra un mensaje de alerta.

Crea un nuevo objeto `Libro` con la información introducida y lo guarda en `LibroDAO`, muestra un mensaje de éxito y los campos se vacían para poder llevar a cabo otro registro.

```
@FXML
void registrarLibro(ActionEvent event) {
    String isbn = tfISBN.getText().trim();
    String titulo = tfTitulo.getText().trim();
    String autor = tfAutor.getText().trim();

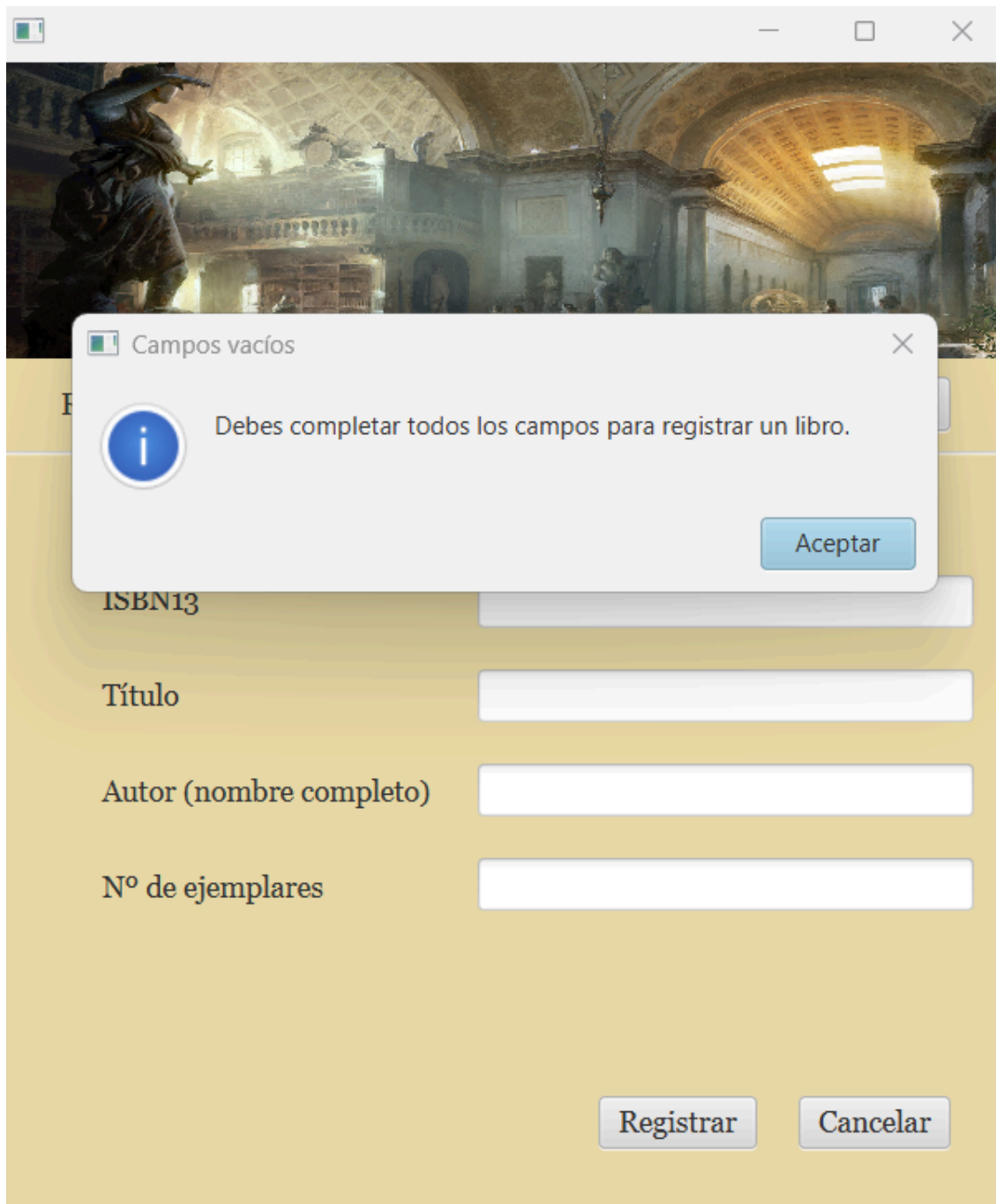
    if (isbn.isEmpty() || titulo.isEmpty() || autor.isEmpty()) {
        mostrarAlerta("Campos vacíos", "Debes completar todos los campos para registrar un libro.");
        return;
    }

    Libro libro = new Libro();
    libro.setIsbn(isbn);
    libro.setTitulo(titulo);
    libro.setAutor(autor);

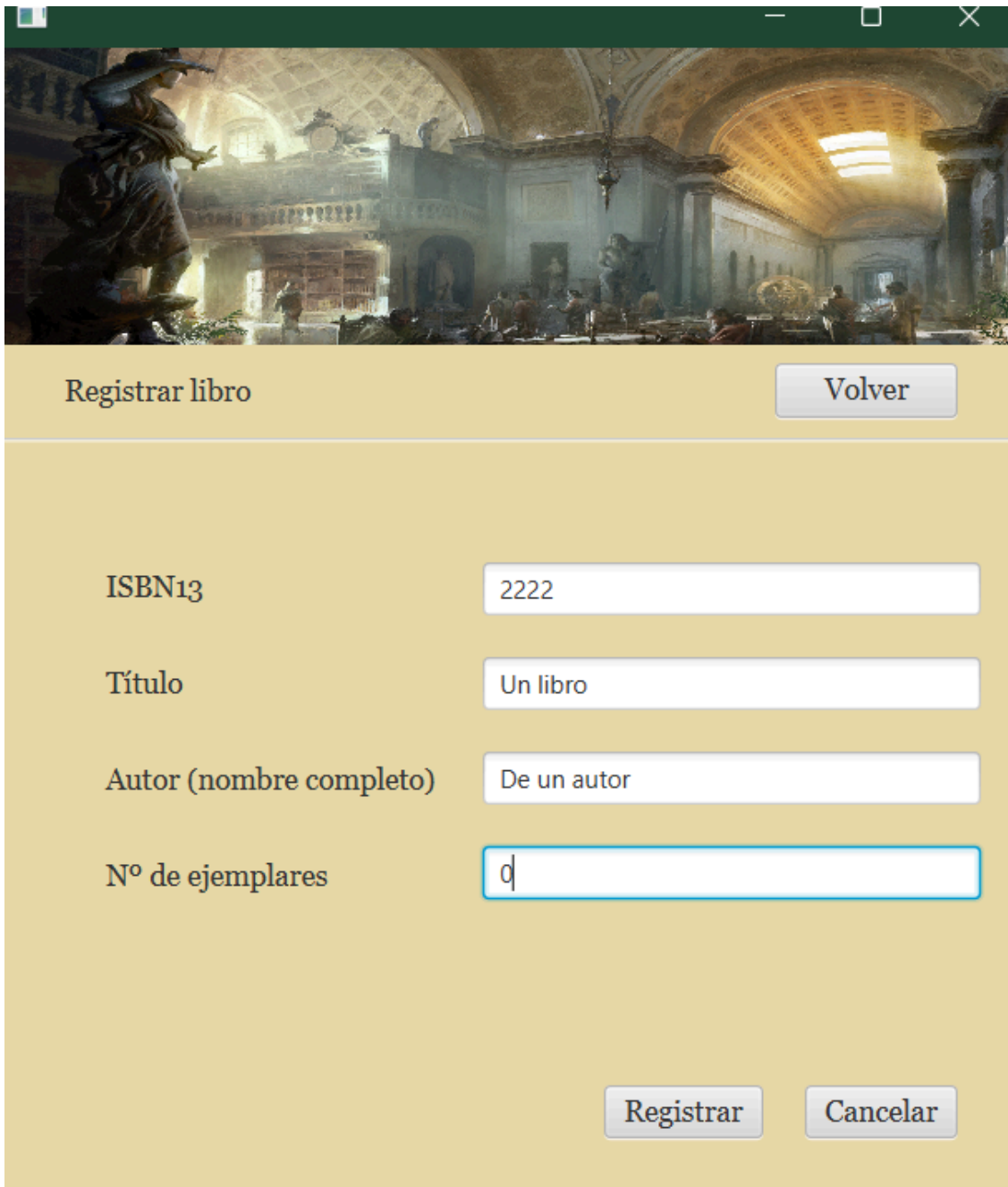
    libroDAO.guardar(libro);

    mostrarAlerta("Éxito", "Libro registrado correctamente.");

    tfISBN.clear();
    tfTitulo.clear();
    tfAutor.clear();
    tfNumeroEjemplares.clear();
}
```



Vamos a registrar un nuevo libro pero sin ejemplares:



Registrar libro

Volver

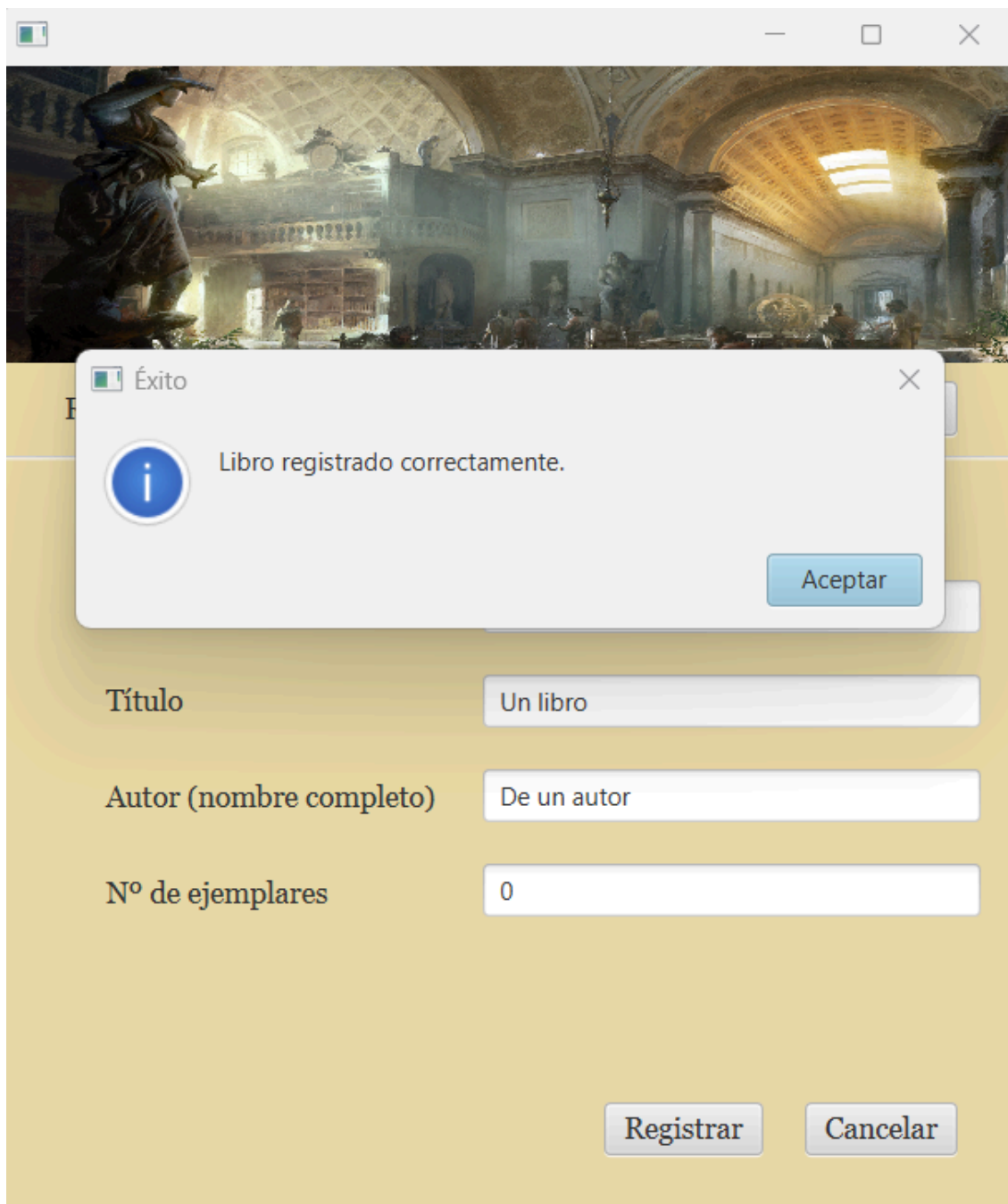
ISBN13 2222

Título Un libro

Autor (nombre completo) De un autor

Nº de ejemplares 0

Registrar Cancelar



Éxito

Libro registrado correctamente.

Aceptar

Título

Un libro

Autor (nombre completo)

De un autor

Nº de ejemplares

0

Registrar

Cancelar



Inventario libros:

Volver

Stock	Estado	ISBN	Autor
0	Sin ejemplares	2222	De un autor
0	Sin ejemplares	22220	De un autor
0	Sin ejemplares	3226558874122585	Homero
1	jj, Disponible	8469465	Homero
2	Disponible	9781111111111	George Orwell
3	Disponible	9781234567890	Miguel de Cervantes
1	Disponible, Dañado	9789876543210	Gabriel García Márquez

Registrar ejemplares de un libro:

Esta clase permite al administrador agregar nuevos ejemplares a los libros existentes en la biblioteca, indicando su estado y asociándolos al libro correspondiente mediante el ISBN.

El método `registrarEjemplares(ActionEvent event)` gestiona el proceso de registro de un nuevo ejemplar, obtiene los datos introducidos por el administrador: ISBN del libro y estado del ejemplar.

Se comprueba que el estado introducido sea válido (`disponible` , `prestado` o `dañado`). Si no lo es, muestra un mensaje de error.

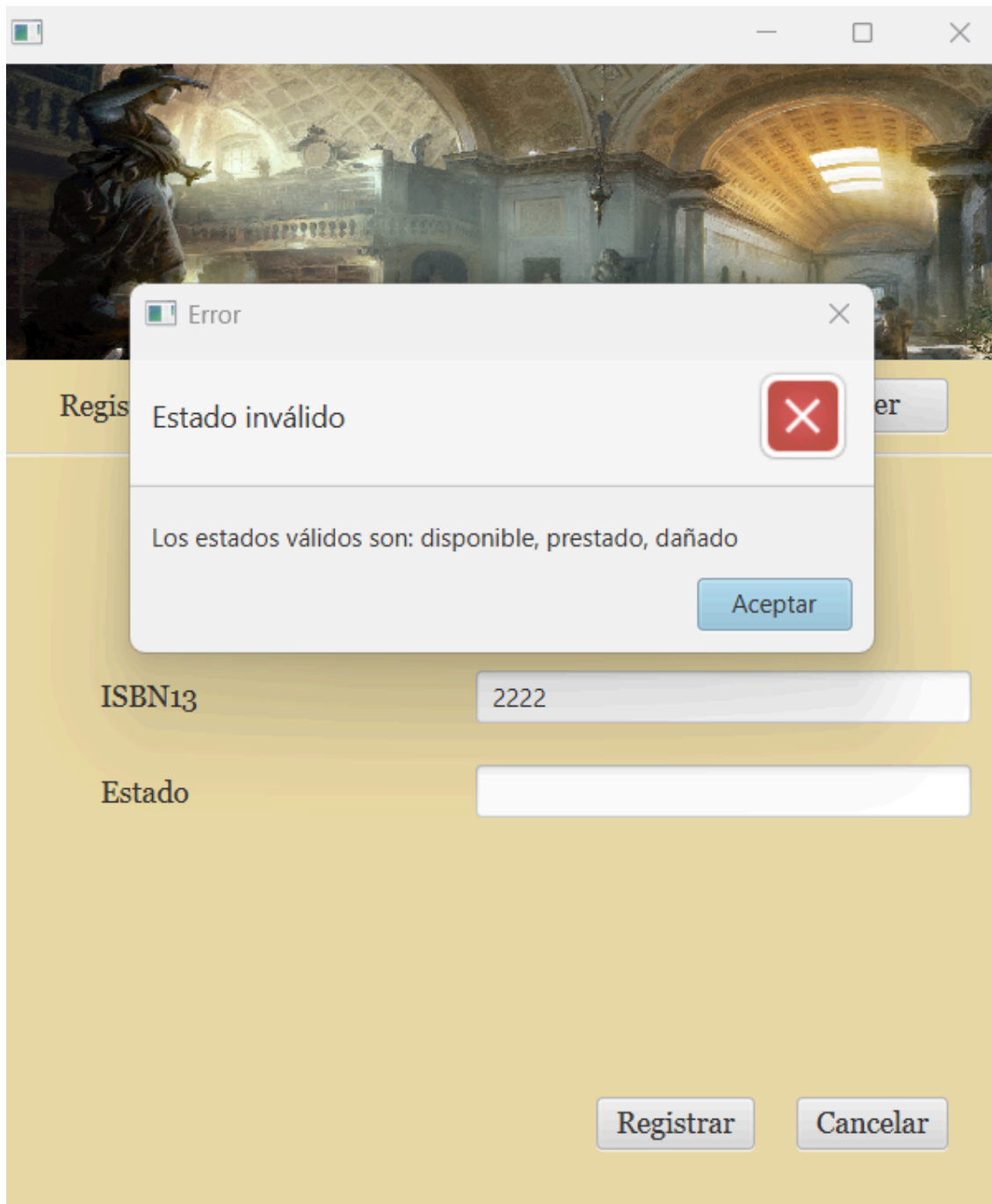
Busca el libro correspondiente mediante `LibroDAO` . Si no existe un libro con ese ISBN, se muestra un mensaje de error.

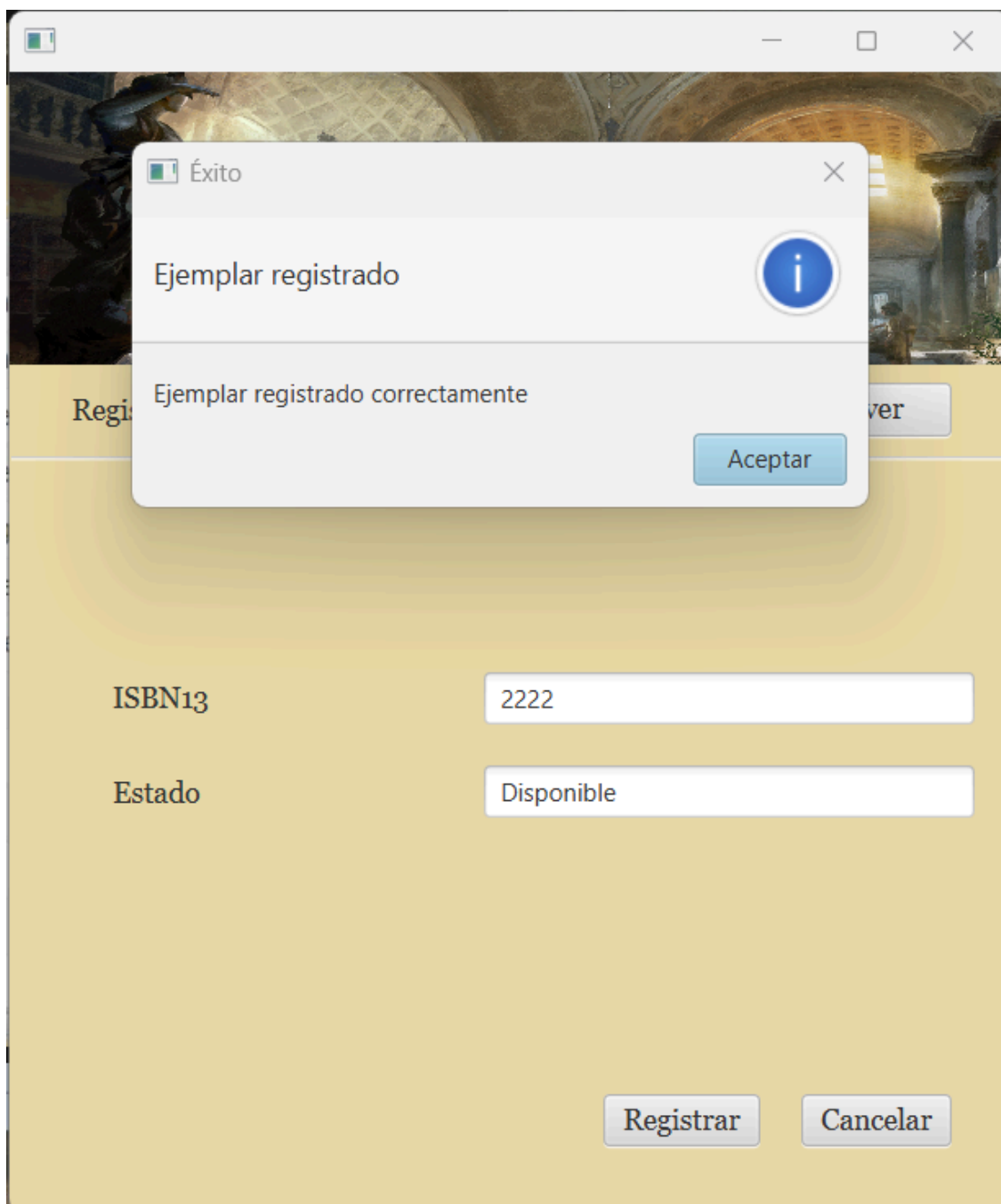
Crea un objeto `Ejemplar` con la información proporcionada y lo guarda en la base de datos mediante `EjemplarDAO` .

Muestra un mensaje confirmando que el ejemplar ha sido registrado correctamente y limpia los campos del formulario para un nuevo registro.

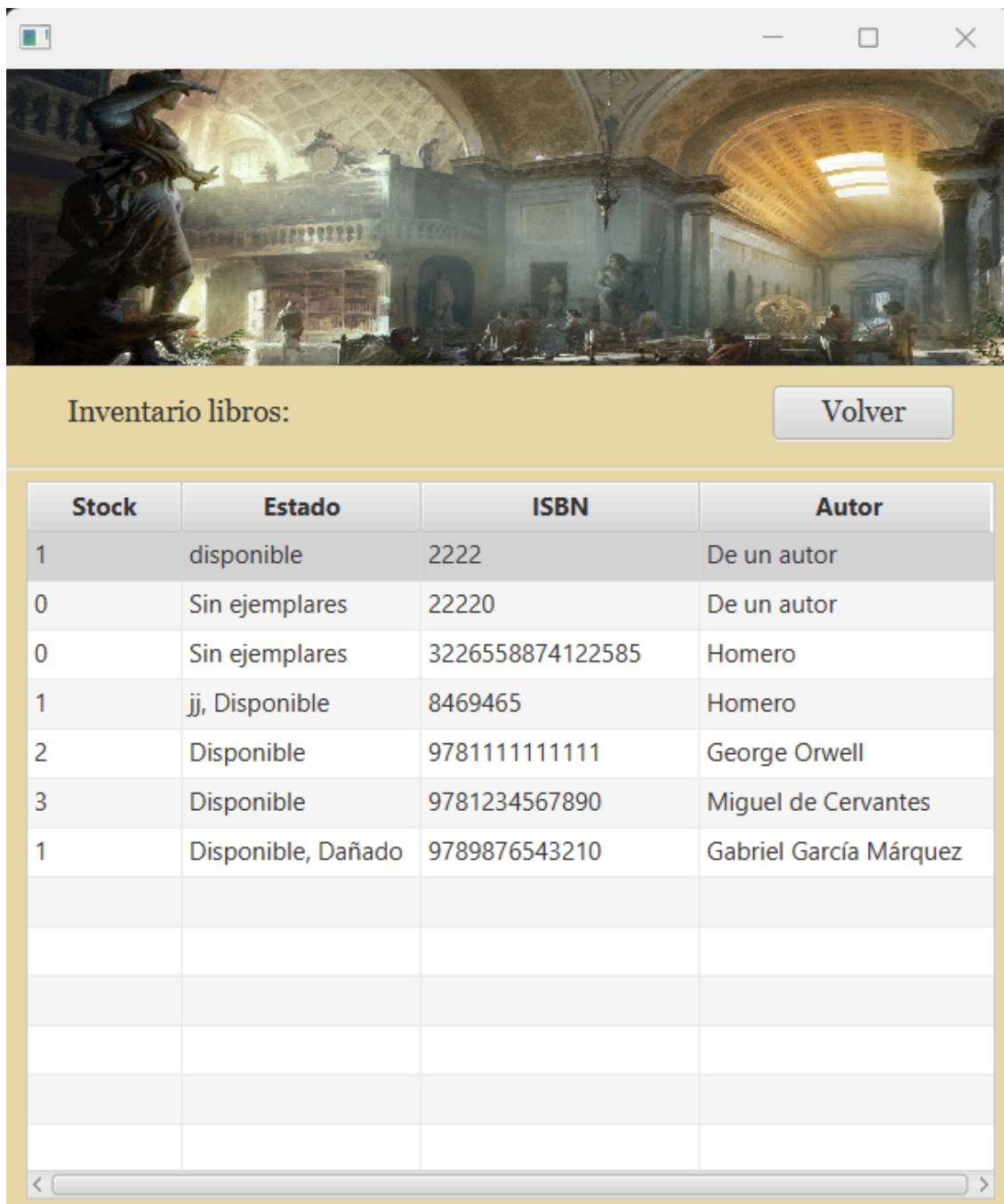
@FXML

```
void registrarEjemplares(ActionEvent event) {  
    String estado = tfEstado.getText().trim().toLowerCase();  
    String isbn = tfISBN.getText().trim();  
  
    List<String> estadosValidos = List.of("disponible", "prestado", "dañado");  
    if (!estadosValidos.contains(estado)) {  
        Alert alert = new Alert(Alert.AlertType.ERROR);  
        alert.setTitle("Error");  
        alert.setHeaderText("Estado inválido");  
        alert.setContentText("Los estados válidos son: disponible, prestado, dañado");  
        alert.showAndWait();  
        return;  
    }  
  
    Libro libro = new LibroDAO().buscarPorISBN(isbn);  
    if (libro == null) {  
        Alert alert = new Alert(Alert.AlertType.ERROR);  
        alert.setTitle("Error");  
        alert.setHeaderText("Libro no encontrado");  
        alert.setContentText("No existe ningún libro con ese ISBN");  
        alert.showAndWait();  
        return;  
    }  
  
    Ejemplar e = new Ejemplar();  
    e.setEstado(estado);  
    e.setIsbn(libro);  
  
    new EjemplarDAO().guardar(e);  
  
    Alert alert = new Alert(Alert.AlertType.INFORMATION);  
    alert.setTitle("Éxito");  
    alert.setHeaderText("Ejemplar registrado");  
    alert.setContentText("Ejemplar registrado correctamente");  
    alert.showAndWait();  
  
    tfEstado.clear();  
    tfISBN.clear();  
}
```





Como podemos ver, el stock del libro con ISBN 2222 es 1:



Stock	Estado	ISBN	Autor
1	disponible	2222	De un autor
0	Sin ejemplares	22220	De un autor
0	Sin ejemplares	3226558874122585	Homero
1	jj, Disponible	8469465	Homero
2	Disponible	9781111111111	George Orwell
3	Disponible	9781234567890	Miguel de Cervantes
1	Disponible, Dañado	9789876543210	Gabriel García Márquez

Lista de préstamos:

Esta clase permite al administrador consultar todos los préstamos realizados en la biblioteca, tanto activos como devueltos.

En el método `initialize()` se definen las columnas del `TableView`, ID del préstamo, ID del usuario, ID del ejemplar, fecha de inicio, fecha de devolución

Cada columna se vincula con la propiedad correspondiente del objeto `Prestamo`, asegurando que la tabla muestre información precisa y actualizada.

```
@FXML
public void initialize() {
    colIdPrestamo.setCellValueFactory( CellDataFeatures<Prestamo, Integer> cell -> new SimpleIntegerProperty(cell.getValue().getId()).asObject());
    colIdUsuario.setCellValueFactory( CellDataFeatures<Prestamo, Integer> cell -> new SimpleIntegerProperty(cell.getValue().getUsuario().getId()).asObject());
    colIdEjemplar.setCellValueFactory( CellDataFeatures<Prestamo, Integer> cell -> new SimpleIntegerProperty(cell.getValue().getEjemplar().getId()).asObject());
    colFechaInicio.setCellValueFactory( CellDataFeatures<Prestamo, LocalDate> cell -> new SimpleObjectProperty<>(cell.getValue().getFechaInicio()));
    colFechaDevolucion.setCellValueFactory( CellDataFeatures<Prestamo, LocalDate> cell -> new SimpleObjectProperty<>(cell.getValue().getFechaDevolucion()));

    cargarPrestamos();
}
```

El método `cargarPrestamos()` obtiene la lista completa de préstamos mediante `PrestamoDAO` y la convierte en una lista observable para mostrarla en la tabla.

Esto permite que cualquier cambio en la base de datos, como la creación de nuevos préstamos o devoluciones, se refleje al recargar la pantalla.

```
private void cargarPrestamos() { 1 usage
    List<Prestamo> prestamos = prestamoDAO.findAll();
    tablaPrestamosUsuario.setItems(FXCollections.observableArrayList(prestamos));
}
```



Lista de préstamos activos:

Volver

ID Préstamo	ID Ejemplar	ID Usuario	Fecha Inicio	Fecha Devolución
1	3	1	2024-11-01	2026-01-06
2	4	2	2024-11-05	2024-11-20
3	7	1	2026-01-06	2026-01-06
4	9	5	2026-01-06	2026-01-06
5	7	5	2026-01-06	2026-01-06
6	6	5	2026-01-06	2026-01-06
7	4	5	2026-01-06	2026-01-06
8	9	5	2026-01-06	2026-01-06
9	7	5	2026-01-06	2026-01-06

Trabajo realizado por: María Buzarhon Navarro - 2 DAM.